Projet : Requêtage de base de données Neo4j à partir de texte en langage naturel via des grands modèles de langage

1 Introduction

Dans ce projet, qui sera réalisé en groupes de 3 étudiants, on explorera comment pouvoir interagir avec nos données de Neo4j en local comme avec un ami : poser des questions en langage naturel et récupérer des informations grâce aux grands modèles de langage (*LLM* pour plus court).

Une telle interaction avec un LLM est uniquement possible, si le modèle est capable de générer de bonnes requêtes Cypher associées aux questions en langage naturel pour intéroger une base de données en local. Par exemple, si l'utilisateur demande de lister les noms de tous les films de *Tom Hanks*, la requête Cypher correspondante sera *MATCH (a :Actor {name : "Tom Hanks"})-[:ACTED_IN]->(m :Movie) RETURN m.title*.

Utiliser un LLM pour générer une telle requête Cypher à partir d'une question en langage naturel présente deux avantages concrets pour les professionnels du monde de l'analyse de données et de la science des données. Premièrement, c'est le gain de temps. Les LLMs peuvent interpréter une question en langage naturel et produire instantanément une requête efficace. Deuxièmement, c'est l'accessibilité. Tous les professionnels ne maîtrisent pas le Cypher/Neo4j, une compétence technique qui peut être intimidante pour ceux qui ne viennent pas du domaine de l'informatique. Grâce aux LLMs, n'importe qui avec une question pertinente peut obtenir une réponse sous forme de requête Cypher sans avoir à se plonger dans les complexités du langage.

Cependant, il y a des limitations à considérer. Même si les LLMs sont performants, ils ne remplacent pas totalement les compétences de codage. Pour des requêtes complexes, notamment celles nécessitant une optimisation fine ou un ajustement spécifique au schéma de la base de données, une compréhension approfondie du Cypher reste essentielle. Il est donc prudent de voir ces outils comme des aides, et non comme des substituts à la compétence humaine. D'ailleurs, on se servira du modèle médiocre *llama 3.2 :3b* sur CPU pour ce projet (i.e. relativement moins puissant par rapport aux autres moèdles), ce qui augmenterait la possibilité d'erreur dans les résultats. Pourtant, ce projet est assez générique, dans le sens où on peut facilement remplacer le modèle *llama 3.2 :3b* par un autre plus puissant pour améliorer la performance.

2 Chargement de données

Vous avez déjà travaillé sur les données de cinéma pendant le dernier TP de Neo4j. Dans ce projet, on se basera sur une petite échantillon de ces données-là, qui sont disponible sous forme d'un fichier en zip sur Moodle. Rappelez-vous que le dataset contient les ratings pour les films. En plus de ça, les données sont enrichies avec des commentaires des utilisateurs sur les films, en plus des rating.

Dans le même document zip, un fichier python vous est fourni pour insérer les données dans votre base Neo4j. Le schéma du graphe est illustrée dans la Figure 1.

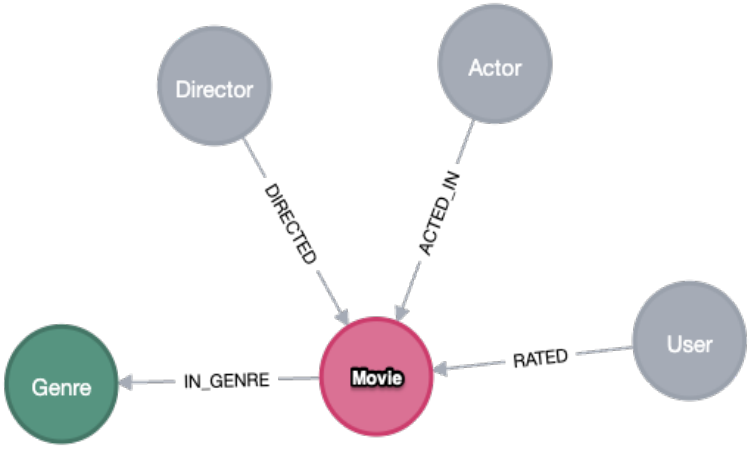


Figure 1 – Schéma du graphe dans la base de données Neo4j.

3 Requêtes Cypher (5 points)

L'utilisation de quelques exemples spécifiques auparavant pour entraîner un modèle de LLM s'avère être très pratique et idéale lorsqu'il a besoin d'aide pour comprendre ce qu'on lui demande de faire. On appelle cela l'apprentissage à la volée (Few Shot Learning en anglais). Ces exemples sont donnés au modèle dans le prompt directement.

Pour qu'un grand modèle de langage puisse proposer correctement une requête Cypher pour une question donnée, vous appliquerez cette méthodologie dans le projet. C'est la raison pour laquelle vous devez préparer en amont certaines questions et leurs requête Cypher correspondantes. Pour rappel, vous avez déjà répondu, entre autres, aux 5 questions suivantes lors du TP de Neo4j

— Quels sont les 5 films les plus surévalues ?

— Quels sont les acteurs des films qui ont plus de rating 9 ?

— Lister les films qui a au moins 2 direct réalisateurs, et rationnellement les noms des personnes qui ont donné leurs avis sur ces films

— lister tous les films de Tom Hanks :

— lister le nombre de co-acteurs de Tom Hanks

Ces questions-là sont déja mis dans le fichier *movie_example_cypher.py*. De plus, il serait mieux d'ajouter plus de questions, accompagnées de leurs requêtes Cypher. C'est la raison pour laquelle on vous liste ci-dessous quelques quelques questions en anglais, et il faut trouver les requêtes Cypher correspondantes.

1. What is the overall sentiment analysis regarding the acting of Robert de Niro ? Keep it as one word.
2. What is the average rating value for the movie Sounder ?
3. What is the most recent movie ?
4. Which movies are not reviewed by Robert Brooks ?
5. What are the co-actors of Robert de Niro, grouped by movie ?
6. List the directors who directed more than one movie.

7. List the American directors and actors
8. List the directors older than 80 years old
9. List the died directors
10. Query the database to list romantic movies
11. Which movie genre gets the most ratings by the users ?
12. Which users co-reviewed the same movies more than twice ?
13. What is the proportion of movies reviewed in common by 'Darlene Garcia' and 'Robert Brooks', i.e. review similarity (réponse : 0.6)
14. What is the most similar movie to the movie "Hollow Man" in terms of movie genre similarity ? (Réponse : "Brazil")
15. Which movie do you recommend to User Robert Brooks based on the existing good ratings ? You need to prioritize the movie which gets the most reviews with rating>3.0. (réponse : the movie Sounder)

4 Langchain (10 points)

Pour rappel, l'appel d'outil dans les LLM est la capacité du modèle à interagir avec des outils, services ou API externes pour effectuer des tâches. Cela permet aux LLM d'étendre leurs fonctionnalités et d'améliorer leur capacité à gérer des tâches du monde réel qui peuvent nécessiter l'accès à des données externes, à des informations en temps réel ou à des applications spécifiques.

Dans ce projet, vous devez implémenter un agent de LLM, qui possède 4 outils (illustrés dans la Figure 2), comme vu dans le tutoriel : 1) exploration des avis de films dans la base Chroma, 2) exploration des informations spécifiques sur les films et les castings depuis la base Neo4j, 3) calcul de similarité entre 2 concepts (e.g. similarité d'avis entre 2 utilisateurs) et 4) recommandation d'un film à un utilisateur.

Pour le premier outil, il faut implémenter une chaine, nommé *review_chain*, qui permet d'explorer des avis de films dans la base Chroma. Pour le deuxième outil, il faut implémenter une chaine combinée : d'abord une première chaine, nommé *cypher_chain*, qui permet de traduire une question textuelle en requête Cypher et ensuite une deuxième chaine, nommé *qa_chain*, qui prend en entrée une requête Cypher, récupère son résultat en l'exécutant et produit une réponse textuelle qui présente le résultat obtenu.

En ce qui concerne chacun des troisième et quatrième outils, il faut implémenter une fonction python, qui prend en paramètre le nom de concept et construit une requête depuis un brouillon bien définie et l'exécute dans la base Neo4j pour obtenir le résultat. Étant donné qu'il est difficile de faire faire ces tâches à un LLM avec le deuxième outil, on passe par des fonctions python comme une solution.

Remarque 1 : Dans la chaine *cypher_chain*, il faut vérifier le syntaxe Cypher proposé par un LLM et corriger ce syntaxe, si possible. Le code pour éventuellement corriger une requête Cypher proposée par un LLM (e.g. le sens de la flèche pour la direction d'une relation, etc.) est founi dans le code de départ (*cypher_utils.py*)

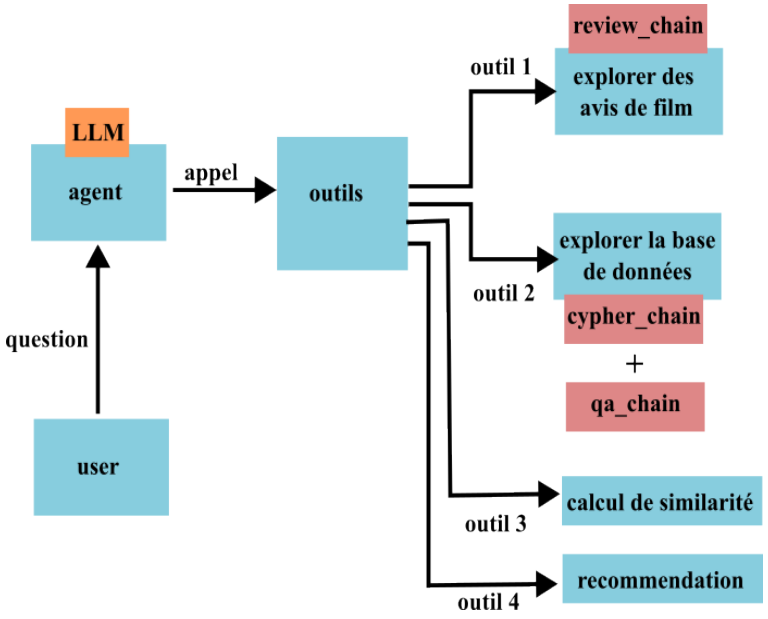


Figure 2 – agent tool calling

Remarque 2 : Dans la chaine *cypher_chain*, il faut donner quelques exemples de questions et leurs requêtes correspondantes au modèle dans le prompt directement (i.e. l'apprentissage à la volée) en utilisant un système de RAG (i.e. des exemples extraits depuis RAG), comme vu dans le tutoriel.

Remarque 3 : Dans la chaine *cypher_chain*, il serait une bonne idée de fournir au modèle LLM le schéma de la base de données. Afin de simplifier votre travail, on vous fournit ce schéma dans le fichier *neo4j_graph_schema.json*, qui est sur Moodle.

Remarque 4 : Il faut créer une interface graphique, comme vu dans le tutoriel, pour interagir avec votre agent. Pour cela, il faut également implémenter l'API de serveur avec FastAPI. Là, il faut créer le chemin "*/movie-rag-agent*" de telle manière que cet URL *"http ://localhost :8000/movie-rag-agent*" nous renvoie la réponse à notre question.

5 Tests (5 points)

Vous devez tester les 2 outils de votre agent avec des tests en batch en mode synchrone, comme vu dans le tutoriel. On vous demande d'inclure les 15 questions suivantes (0.33 point par question) dans ce test et puis enregistrer les résultats dans un fichier de texte :

1. What is the overall sentiment analysis regarding the acting of Jane Fonda ? Keep it as one word.
2. What is the average rating value for the movie Hollow Man ?
3. What is the oldest movie ?
4. Which movies are not reviewed by Darlene Garcia ?

5. What are the co-actors of Kim Dickens, grouped by movie ?
6. List the directors who directed only once.
7. List the British directors and actors
8. List the directors younger than 60 years old
9. List the alive directors
10. Query the database to list comedy movies
11. Which movie genre gets the least ratings by the users ?
12. Which users never co-reviewed together ?
13. What is the proportion of movies reviewed in common by Darlene Garcia and Elizabeth Gonzalez, i.e. review similarity
14. What is the most similar movie to the movie Stanley & Iris in terms of movie genre similarity ?
15. Which movie do you recommend to user Darlene Garcia based on the existing good ratings ? You need to prioritize the movie which gets the most reviews with rating>3.0.

6 Bonus (2 points)

Si vous avez encore le temps, vous pouvez essayer de remplacer les 3ème (similarité) et 4ème (recommendation) outils par le 2ème outil en ajoutant des exemples dans la base Chroma.

7 Rendu

Il faut déposer sur Moodle les élements suivants :

— un fichier texte contenant votre projet sur Gitlab

— le fichier contenant les résultats à votre test en batch

— rapport